

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO

MÔN: HỆ NHÚNG

**Project: Game Bắn trứng khủng long sử dụng
TouchGFX**

Sinh viên thực hiện : Nhóm OK VIP
Mã lớp : 168505
Giáo viên hướng dẫn: TS. Ngô Lam Trung

Hà Nội, tháng 07 năm 2026

I. Giới thiệu đề tài:	2
1. Lý do chọn đề tài:	2
2. Các chức năng của hệ thống:	2
II. Phân tích hệ thống:	3
1. Sơ đồ khối của hệ thống:	3
2. Vai trò từng khối trong hệ thống:	3
III. Thiết kế hệ thống:	4
1. Sơ đồ mạch chi tiết:	4
2. Mô tả và giải thích phương pháp thiết kế:	5
3. Giải thích hoạt động phần mềm:	5
IV. Kết quả:	11
1. Hình ảnh thực tế của hệ thống:	11
2. Demo:	12
V. Kết luận:	15
1. Ưu điểm/ Nhược điểm của hệ thống:	15
2. Giải pháp đề xuất trong tương lai:	16
VI. Tài liệu tham khảo:	16
VII. Phụ lục:	17

Phần 1: Phân công nhiệm vụ:

MSSV	Tên thành viên	Nhiệm vụ
20235347	Trần Ngọc Huyền	Thiết kế giao diện + Tích hợp âm thanh cho hệ thống
20235362	Nguyễn Bảo Linh	Logic game + kết nối joystick

Phần 2: Nội dung đề tài:

I. Giới thiệu đề tài:

1. Lý do chọn đề tài:

Trong chương trình học về hệ thống nhúng, sinh viên được học các kiến thức về vi điều khiển STM32, lập trình ngoại vi và giao tiếp với các thiết bị phần cứng. Để vận dụng những kiến thức đã học vào thực tế, nhóm lựa chọn đề tài "Game bắn trúng khủng long trên STM32 sử dụng joystick và loa 8Ω".

Đề tài giúp nhóm thực hành lập trình trên vi điều khiển STM32, đọc tín hiệu từ joystick để điều khiển hướng bóng, hiển thị hình ảnh trên màn hình và phát âm thanh thông qua loa. Bên cạnh đó, quá trình xây dựng trò chơi còn giúp nhóm hiểu rõ hơn về cách phối hợp giữa phần cứng và phần mềm trong một hệ thống nhúng.

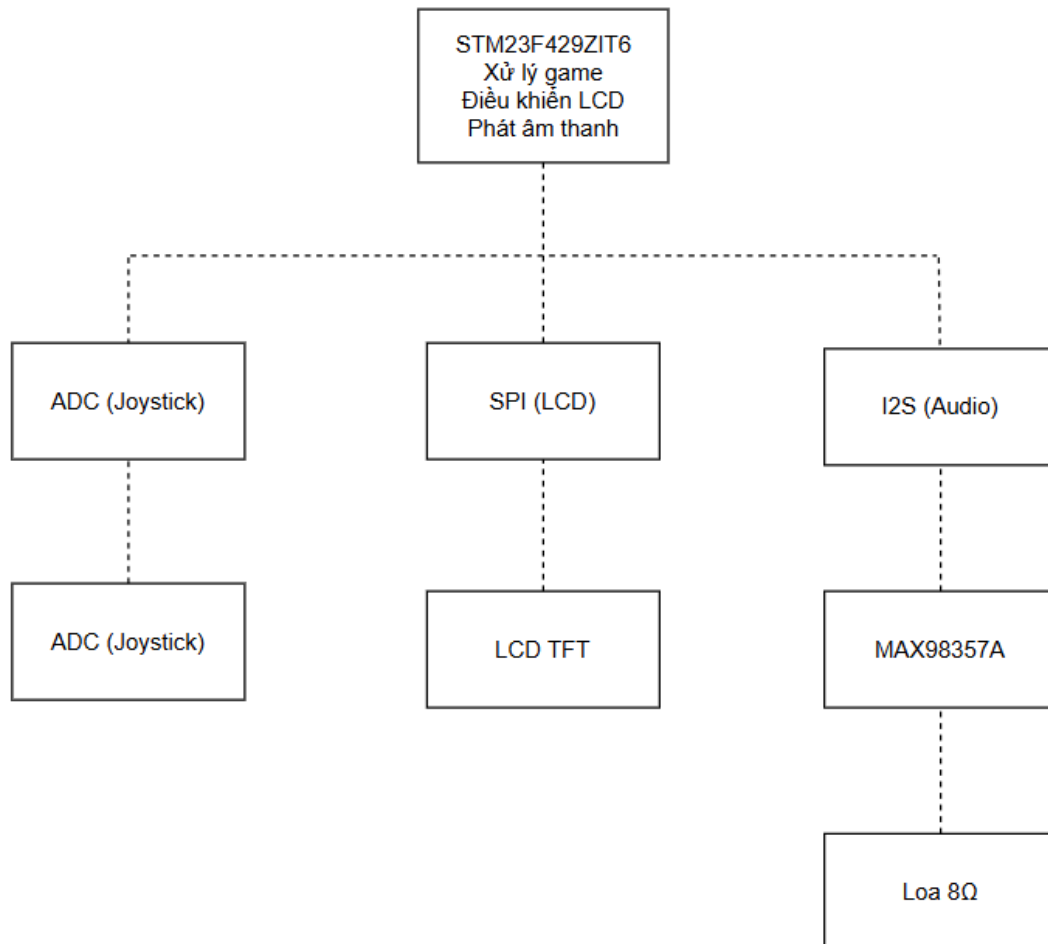
Ngoài việc đáp ứng yêu cầu của môn học, trò chơi còn tạo hứng thú trong quá trình học tập vì có tính thực quan, dễ kiểm tra kết quả và giúp rèn luyện kỹ năng lập trình, tư duy logic cũng như làm việc nhóm. Qua đề tài này, nhóm có cơ hội củng cố kiến thức đã học và nâng cao khả năng phát triển các ứng dụng nhúng trong thực tế.

2. Các chức năng của hệ thống:

Màn hình LCD trên kit STM32 hiển thị hình ảnh và hiệu ứng của trò chơi. Có thể sử dụng cảm ứng của màn hình hoặc Joystick/nút bấm để điều khiển hướng bắn. Trò chơi sẽ bắt đầu khi người dùng nhấn nút Start trên màn hình trang chủ. Trò chơi kết thúc khi người chơi để bóng chạm vạch (dây đỏ) trên màn hình.

II. Phân tích hệ thống:

1. Sơ đồ khối của hệ thống:



2. Vai trò từng khối trong hệ thống:

a. Khối điều khiển trung tâm (STM32):

STM32F429ZIT6 được lựa chọn vì sở hữu lõi ARM Cortex-M4 tốc độ 180 MHz, tích hợp bộ xử lý dấu chấm động (FPU), bộ nhớ Flash lớn và nhiều ngoại vi hỗ trợ đồ họa, âm thanh.

- Trong hệ thống, STM32 đảm nhiệm các chức năng:
- Đọc giá trị trục X, Y của joystick thông qua bộ chuyển đổi ADC.
- Đọc trạng thái nút nhấn trên joystick.
- Xử lý thuật toán game (di chuyển súng, tạo bóng, kiểm tra va chạm, tính điểm).
- Điều khiển hiển thị giao diện trên màn hình LCD.
- Phát hiệu ứng âm thanh qua giao tiếp I2S.

Việc sử dụng STM32F429 giúp hệ thống xử lý đồng thời nhiều tác vụ với độ trễ thấp, đáp ứng yêu cầu thời gian thực của trò chơi.

b. Khối Joystick:

Joystick được sử dụng để điều khiển hướng bắn và thực hiện thao tác xác nhận.

Joystick gồm:

- Hai biến trở tương ứng với trục X và Y.
- Một nút nhấn tích hợp.
- Hai biến trở tạo ra điện áp thay đổi theo góc nghiêng của cần điều khiển. STM32 sử dụng hai kênh ADC để chuyển đổi điện áp này thành giá trị số, từ đó xác định hướng điều khiển.

Nút nhấn của joystick được kết nối tới chân GPIO và đọc ở chế độ Digital Input để thực hiện thao tác bắn bóng.

Ưu điểm của joystick:

- Điều khiển trực quan.
- Độ nhạy cao.
- Chi phí thấp.
- Dễ kết nối với STM32.

c. Khối Audio:

Để tạo hiệu ứng âm thanh cho trò chơi, hệ thống sử dụng module MAX98357A kết hợp với loa 8 Ω .

Khác với các mạch khuếch đại analog truyền thống, MAX98357A tích hợp:

- Bộ giải mã âm thanh số (DAC).
- Bộ khuếch đại Class D.
- Giao tiếp I2S.

Điều này cho phép STM32 truyền trực tiếp dữ liệu âm thanh số mà không cần DAC ngoài. MAX98357A chỉ cần nhận ba tín hiệu:

- BCLK (Bit Clock)
- LRCLK (Word Select)
- DIN (Data)

Sau đó IC tự động chuyển đổi dữ liệu I2S thành tín hiệu công suất đủ lớn để điều khiển loa. MAX98357A hỗ trợ nguồn từ 2,5 V đến 5,5 V, hiệu suất khoảng 92% khi tải 8 Ω và không yêu cầu xung MCLK, giúp đơn giản hóa thiết kế phần cứng

III. Thiết kế hệ thống:

1. Sơ đồ mạch chi tiết:

Module	Chân trên module	Chân trên STM32F429ZIT6
Joystick	VCC	5V
	GND	GND
	X	PC3 (ADC1 IN13)
	Y	PA5 (AD2 IN5)

	SW	PC13
MAX98357A	LRC	PA15
	BCLK	PC10
	DIN	PC12
	GND	GND
	VIN	5V
Loa 8Ohm	Cực dương	Chân + trên MAX98357A
	Cực dương	Chân - trên MAX 98357A

2. Mô tả và giải thích phương pháp thiết kế:

Khi người chơi di chuyển joystick, STM32 liên tục đọc giá trị ADC để xác định hướng ngắ. Khi nhấn joystick hoặc bấm nút “USER”, vi điều khiển thực hiện thuật toán bắn bóng và cập nhật trạng thái trò chơi. Kết quả được hiển thị trên màn hình LCD.

Đồng thời, STM32 phát dữ liệu âm thanh số thông qua giao tiếp I2S tới MAX98357A.

Module này chuyển đổi tín hiệu số thành tín hiệu công suất và truyền trực tiếp tới loa 8 Ω để phát các hiệu ứng âm thanh tương ứng với từng sự kiện trong game.

Nhờ kiến trúc này, hệ thống đảm bảo:

- Tốc độ xử lý nhanh, đáp ứng thời gian thực.
- Âm thanh rõ ràng với số lượng linh kiện tối giản.
- Khả năng mở rộng và bảo trì thuận tiện do các khối chức năng được thiết kế độc lập.

3. Giải thích hoạt động phần mềm:

3.1. Đọc input từ người chơi:

```

void StartDefaultTask(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    JoystickData_t joy;

    for(;;)
    {
        // Đọc VRx
        HAL_ADC_Start(&hadc1);
        HAL_ADC_PollForConversion(&hadc1, 10);
        joy.x = HAL_ADC_GetValue(&hadc1);

        // Đọc VRy
        HAL_ADC_Start(&hadc2);
        HAL_ADC_PollForConversion(&hadc2, 10);
        joy.y = HAL_ADC_GetValue(&hadc2);

        // Đọc nút nhấn SW
        joy.button = ((HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET) ||
                     (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET));

        osMessageQueuePut(myQueue01Handle, &joy, 0, 10);
        osDelay(100);
    }
    /* USER CODE END 5 */
}

```

- Trục X (VRx) - đọc qua ADC1 (channel 13, cấu hình ở MX_ADC1_Init), độ phân giải 8-bit → giá trị 0-255, lưu vào joy.x.
- Trục Y (VRy) - đọc qua ADC2 (channel 5, MX_ADC2_Init), cũng 8-bit, lưu vào joy.y.
- Nút nhấn (SW) - đọc từ 2 chân GPIO:
 - + PC13 (chân nút on-board của board Discovery, cấu hình pull-up, active-low nên check == GPIO_PIN_RESET)
 - + PA0 (input, không pull, check == GPIO_PIN_SET)
- Hai điều kiện này OR - code đang hỗ trợ 2 nguồn nút nhấn khác nhau (có thể là nút on-board và nút của joystick module rồi), bấm cái nào cũng được coi là "pressed".
- Kết quả đóng gói vào struct JoystickData_t (x, y, button - đều uint8_t), rồi đẩy vào myQueue01Handle, một message queue RTOS, để task GUI (TouchGFX) ở nơi khác đọc ra và xử lý logic điều khiển trong game.

3.2. Logic của game:

- a. Tổ chức dữ liệu:

```

enum BubbleColor : uint8_t {
    BC_RED = 0, BC_BLUE, BC_GREEN, BC_YELLOW, BC_PINK, BC_PURPLE, BC_NONE = 0xFF
};

struct Bubble {
    BubbleColor color;
    bool alive;
};

struct FallingBubble
{
    float x;
    float y;
    float vy;

    BubbleColor color;
    bool active;
};

class BubbleGame {
public:
    GameState gs;

    BubbleGame();

    void init(); // Khởi tạo lưới + random bong bóng
    void shootBall(float angle);
    void updateBall(); // Cập nhật bóng đang bay
    void placeBall(int row, int col);
    void cellToPixel(int row, int col, int& px, int& py);
    void pushBoardDown();
    bool isShortRow(int row);
    int getRowSize(int row);
    void updateFalling();
    void spawnFallingBubble(int row, int col);
private:
    bool pixelToCell(int px, int py, int& row, int& col);
    int getNeighbors(int row, int col, int outR[], int outC[]);
    int floodFill(int startR, int startC, BubbleColor color, int resultR[], int resultC[]);
    void dropFloating();
    uint8_t rng8();
};

```

Mà trận bóng được lưu bằng mảng hai chiều thông thường:

Về bản chất, đây không phải một cấu trúc dữ liệu hex chuyên biệt - mỗi hàng vẫn có đủ 9 ô như nhau ở tầng lưu trữ. Cấu trúc so le hoàn toàn được tạo ra ở tầng xử lý logic và hiển thị: đối với các hàng được đánh dấu là "hàng ngắn", ô cuối cùng (cột thứ 9) đơn giản là không được sử dụng.

Cách tiếp cận này được lựa chọn vì đơn giản, dễ triển khai trên vi điều khiển, tránh chi phí bộ nhớ và tính toán của các mô hình tọa độ hex phức tạp hơn.

b. Xác định trạng thái so le của từng hàng:

Việc một hàng là "ngắn" (8 ô) hay "dài" (9 ô) không cố định theo chỉ số hàng tuyệt đối, mà phụ thuộc vào một cờ trạng thái toàn cục:


```

bool BubbleGame::isShortRow(int row)
{
    bool shortRow = ((row & 1) != 0);

    if(gs.topRowOdd)
        shortRow = !shortRow;

    return shortRow;
}

int BubbleGame::getRowSize(int row)
{
    return isShortRow(row) ? (COLS - 1) : COLS;
}

```

Cờ topRowOdd được lưu trong GameState và bị đảo giá trị mỗi khi lưới bị đẩy xuống một hàng (pushBoardDown()). Thiết kế này đảm bảo tính nhất quán của cấu trúc so le: khi một hàng mới được đẩy lên trên cùng, tính chẵn/lẻ vật lý của chỉ số hàng không thay đổi, nhưng để hai hàng liền kề luôn khác kiểu (một ngắn, một dài - đúng bản chất lưới tổ ong), trạng thái tham chiếu phải được lật lại theo mỗi lần dịch chuyển.

c. Chuyển đổi tọa độ lưới sang tọa độ pixel:

```

// Chuyển từ (row, col) => (pixel x, pixel y)
// Đếm 2 bên lề trái phải
// Hàng lẻ sẽ bị lệch nửa ô so với hàng chẵn
void BubbleGame::cellToPixel(int row, int col, int& px, int& py) {
    const int SIDE_MARGIN = 13;
    const int TOP_MARGIN = 23;

    int offsetX = isShortRow(row) ? R_PIX : 0;
    px = SIDE_MARGIN + offsetX + col * (R_PIX * 2) + R_PIX;
    py = TOP_MARGIN + row * (int)(R_PIX * 1.732f) + R_PIX;
}

// Ngược lại so với hàm cellToPixel()
// Dùng để chuyển đổi tọa độ của bóng sau
// khi bắn lên lưới sẽ nằm ở (row, col) nào
bool BubbleGame::pixelToCell(int px, int py, int& row, int& col) {
    const int SIDE_MARGIN = 13;
    const int TOP_MARGIN = 23;

    row = (int)roundf((py - TOP_MARGIN - R_PIX) / (R_PIX * 1.72f));

    if (row < 0) row = 0;
    if (row >= ROWS) row = ROWS - 1;

    int offsetX = isShortRow(row) ? R_PIX : 0;
    col = (int)roundf((px - SIDE_MARGIN - R_PIX - offsetX) / (float)(R_PIX * 2));

    int maxCol = getRowSize(row);
    if (col < 0) col = 0;
    if (col >= maxCol) col = maxCol - 1;
    return true;
}

```

Trục X: Hàng ngắn được cộng thêm bán kính bóng => Tạo độ lệch nửa ô giữa hàng ngắn và hàng dài liền kề

Trục Y: Khoảng cách giữa các hàng là $R_PIX \times 1.732 (\approx R_PIX \times \sqrt{3})$ => Hệ số hình học chuẩn của lưới lục giác đóng gói khít, giúp các hàng "lồng" khít vào nhau.

d. Thuật toán tìm chuỗi bóng cùng màu - BFS:

```
int BubbleGame::floodFill(int startR, int startC, BubbleColor color,
                          int resultR[], int resultC[]) {
    // Visited array
    bool visited[ROWS][COLS];
    memset(visited, 0, sizeof(visited));

    // BFS queue
    int qR[ROWS * COLS], qC[ROWS * COLS];
    int head = 0, tail = 0;
    int count = 0;

    qR[tail] = startR;
    qC[tail] = startC;
    tail++;
    visited[startR][startC] = true;

    while (head < tail) {
        int r = qR[head];
        int c = qC[head];
        head++;

        if (!gs.cells[r][c].alive) continue;
        if (gs.cells[r][c].color != color) continue;

        resultR[count] = r;
        resultC[count] = c;
        count++;

        int nr[6], nc[6];
        int n = getNeighbors(r, c, nr, nc);
        for (int i = 0; i < n; i++) {
            if (!visited[nr[i]][nc[i]]) {
                visited[nr[i]][nc[i]] = true;

                qR[tail] = nr[i];
                qC[tail] = nc[i];
                tail++;
            }
        }
    }
    return count;
}
```

Thuật toán BFS (Breadth-First Search), áp dụng trên đồ thị lưới hex (mỗi ô là 1 đỉnh, cạnh nối tới 6 lân cận). Dùng để tìm tất cả các ô cùng màu, liên thông trực tiếp hoặc gián tiếp với ô vừa đặt bóng - nền tảng của cơ chế "nổ chuỗi ≥ 3 bóng cùng màu".

e. Thuật toán xác định bóng "mò côi" rơi xuống - BFS từ nguồn đa điểm (Multi-source BFS):

```

void BubbleGame::dropFloating() {
    bool connected[ROWS][COLS];
    memset(connected, 0, sizeof(connected));

    // BFS từ hàng 0 – mọi bóng liên thông với hàng 0 là "còn treo"
    int qR[ROWS * COLS], qC[ROWS * COLS];
    int head = 0, tail = 0;

    for (int c = 0; c < COLS; c++) {
        if (gs.cells[0][c].alive && !connected[0][c]) {
            connected[0][c] = true;
            qR[tail] = 0;
            qC[tail] = c;
            tail++;
        }
    }

    while (head < tail) {
        int r = qR[head];
        int c = qC[head];
        head++;

        int nr[6], nc[6];
        int n = getNeighbors(r, c, nr, nc);
        for (int i = 0; i < n; i++) {
            int rr = nr[i], cc = nc[i];
            if (!connected[rr][cc] && gs.cells[rr][cc].alive) {
                connected[rr][cc] = true;
                qR[tail] = rr;
                qC[tail] = cc;
                tail++;
            }
        }
    }
}

// Xóa tất cả bóng không connected
for (int r = 0; r < ROWS; r++) {
    int maxC = getRowSize(r);
    for (int c = 0; c < maxC; c++) {
        if (gs.cells[r][c].alive && !connected[r][c]) {
            spawnFallingBubble(r, c);

            gs.cells[r][c].alive = false;
            gs.score += 5; // bonus điểm bóng rơi
        }
    }
}
}

```

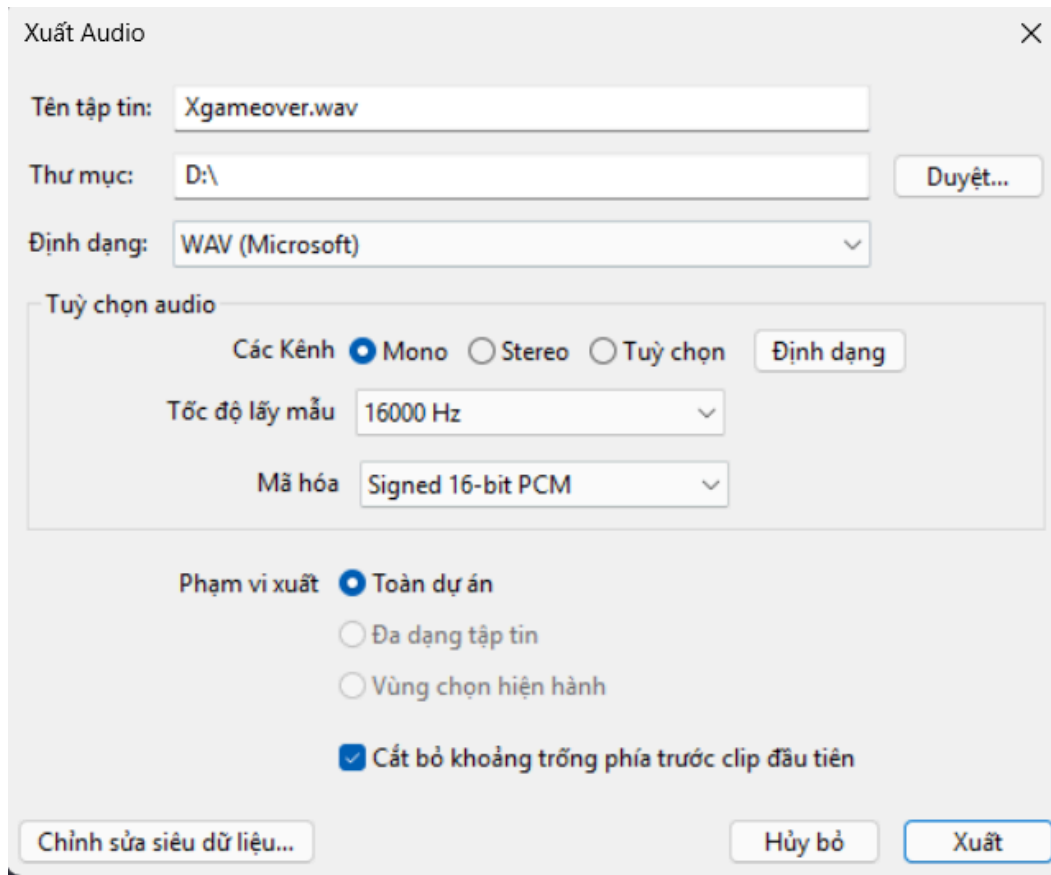
Khởi động BFS đồng thời từ nhiều điểm nguồn (toàn bộ ô ở hàng trên), tìm tập hợp mọi ô còn "liên kết vật lý" với trên. Bất kỳ ô nào bóng sống nhưng nằm ngoài tập hợp connected được coi là mất kết nối vật lý (mô phỏng nguyên lý: bóng chỉ đứng yên nếu có đường nối liên tục về phía trên) và bị loại bỏ.

3.3. Xử lý và tạo âm thanh

Trong dự án, nhóm sử dụng hai phương pháp để tạo và phát hiệu ứng âm thanh nhằm cân bằng giữa chất lượng âm thanh và tài nguyên của vi điều khiển STM32.

- Phương pháp thứ nhất là sử dụng các file âm thanh có sẵn. Các hiệu ứng như BreakEgg và Game Over được tải về hoặc chỉnh sửa bằng phần mềm Audacity, sau đó xuất dưới định dạng WAV (Microsoft) với cấu hình Mono, tần số lấy mẫu 16 kHz và mã hóa Signed 16-bit PCM. Việc lựa chọn cấu hình này giúp dữ liệu âm thanh tương thích trực tiếp với giao tiếp I2S của STM32 và IC khuếch đại MAX98357A, đồng thời giảm dung lượng lưu trữ so với âm thanh Stereo hoặc tần số lấy mẫu cao hơn. Sau khi xuất, file

WAV được chuyển đổi thành mảng dữ liệu (C Array) bằng công cụ chuyển đổi trực tuyến, từ đó tích hợp trực tiếp vào chương trình. Khi cần phát âm thanh, STM32 sử dụng DMA để truyền mảng dữ liệu PCM qua giao tiếp I2S đến IC MAX98357A và xuất ra loa.



Xuất Audio

Tên tập tin: Xgameover.wav

Thư mục: D:\ Duyệt...

Định dạng: WAV (Microsoft)

Tùy chọn audio

Các Kênh ☒ Mono ☐ Stereo ☐ Tùy chọn Định dạng

Tốc độ lấy mẫu 16000 Hz

Mã hóa Signed 16-bit PCM

Phạm vi xuất ☒ Toàn dự án

☐ Đa dạng tập tin

☐ Vùng chọn hiện hành

☒ Cắt bỏ khoảng trống phía trước clip đầu tiên

Chỉnh sửa siêu dữ liệu... Hủy bỏ Xuất

- Phương pháp thứ hai là sinh âm thanh trực tiếp bằng chương trình thông qua hàm sóng sin. Thay vì lưu sẵn dữ liệu âm thanh, chương trình tính toán giá trị của từng mẫu dựa trên công thức của hàm sin và lưu vào bộ đệm (audioBuffer). Bằng cách thay đổi tần số, biên độ và thời gian phát, có thể tạo ra các hiệu ứng như tiếng bắn, tiếng nhấn nút hoặc tiếng vỡ trứng mà không cần sử dụng file âm thanh. Phương pháp này giúp tiết kiệm bộ nhớ Flash, dễ dàng điều chỉnh đặc tính âm thanh và phù hợp với các hiệu ứng ngắn trong trò chơi.

```

int16_t breakBuffer[2000];
int16_t startBuffer[8000];
int16_t gameOverBuffer[16000];
int16_t shootBuffer[1600];
volatile uint8_t audioBusy = 0;

void CreateStartGameSound() {
    int length = 8000;
    float notes[] = {523.25, 659.25, 783.99, 1046.50};
    int note_len = length / 4;
    for (int n = 0; n < 4; n++) {
        for (int i = 0; i < note_len; i++) {
            int idx = n * note_len + i;
            float t = (float)idx / SAMPLING_RATE;
            float sample = 4000.0f * sinf(2.0f * M_PI * notes[n] * t);
            startBuffer[idx] = (uint16_t)((int16_t)sample);
        }
    }
}

void CreateGameOverSound() {
    int length = 16000;
    float notes[] = {392.00, 329.63, 261.63, 196.00};
    int note_len = length / 4;
    for (int n = 0; n < 4; n++) {
        for (int i = 0; i < note_len; i++) {
            int idx = n * note_len + i;
            float t = (float)idx / SAMPLING_RATE;

            float wave = (sinf(2.0f * M_PI * notes[n] * t) > 0) ? 1.0f : -1.0f;
            float envelope = 1.0f - ((float)i / note_len);
            gameOverBuffer[idx] = (uint16_t)((int16_t)(wave * 2000.0f * envelope));
        }
    }
}

void CreateBubbleShootSound()
{
    for(int i = 0; i < 1600; i++)
    {
        float p = (float)i / 1600;
        float t = (float)i / SAMPLING_RATE;

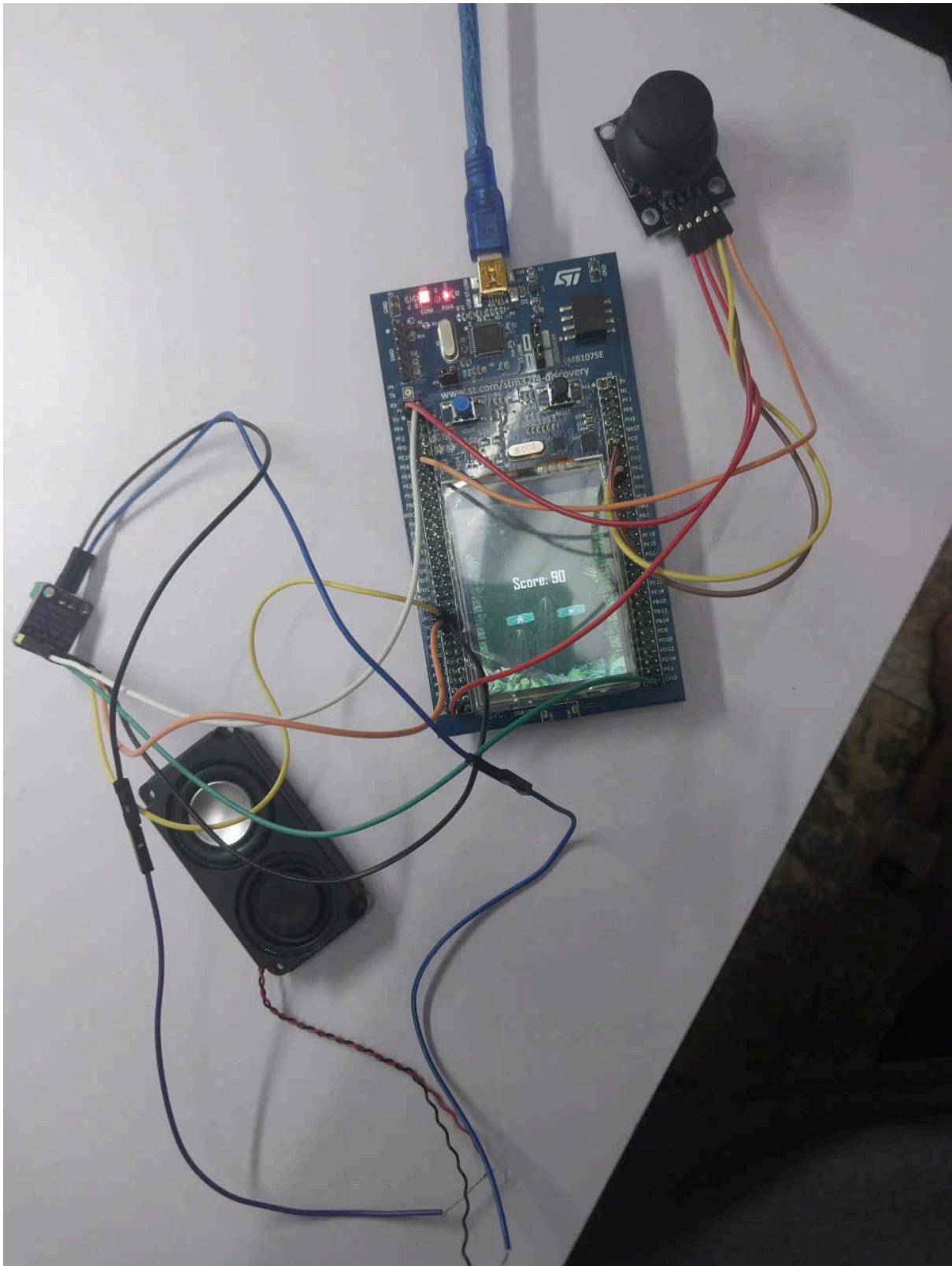
        float freq = 2200.0f - (1500.0f * p);
        float sample = sinf(2.0f * M_PI * freq * t);

        float env = 1.0f - p;
        float volume = 0.3f;
        shootBuffer[i] = (int16_t)(sample * 30000.0f * env*volume);
    }
}

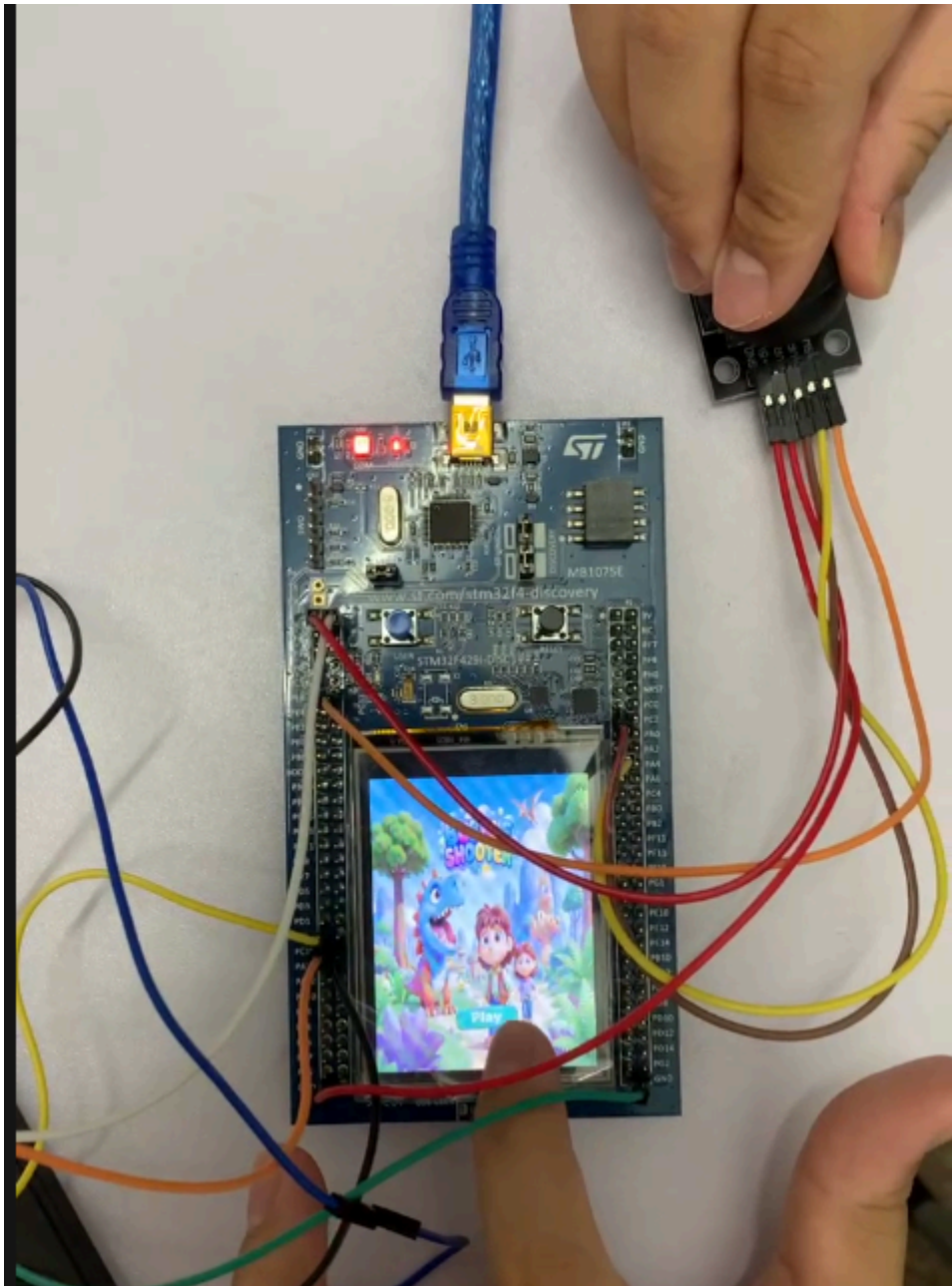
```

IV. Kết quả:

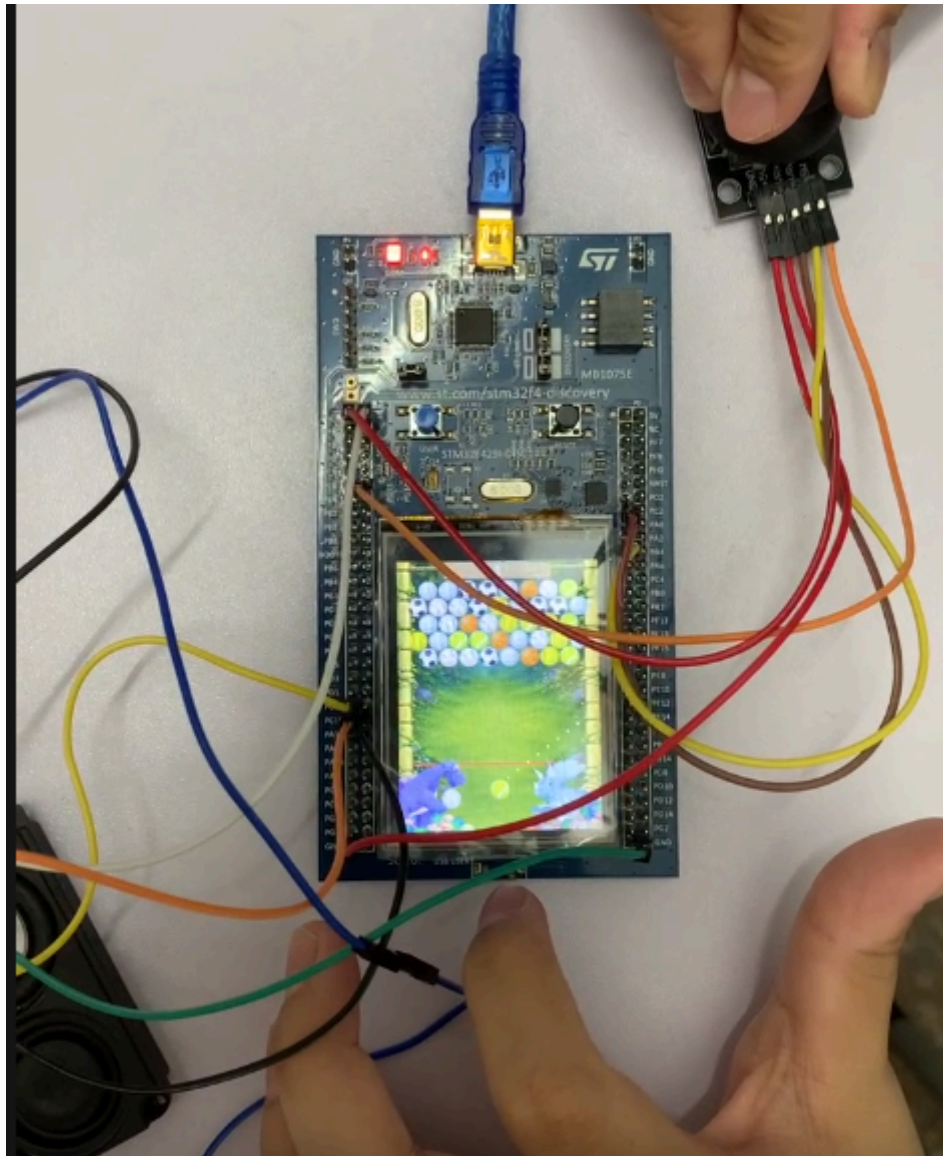
1. Hình ảnh thực tế của hệ thống:



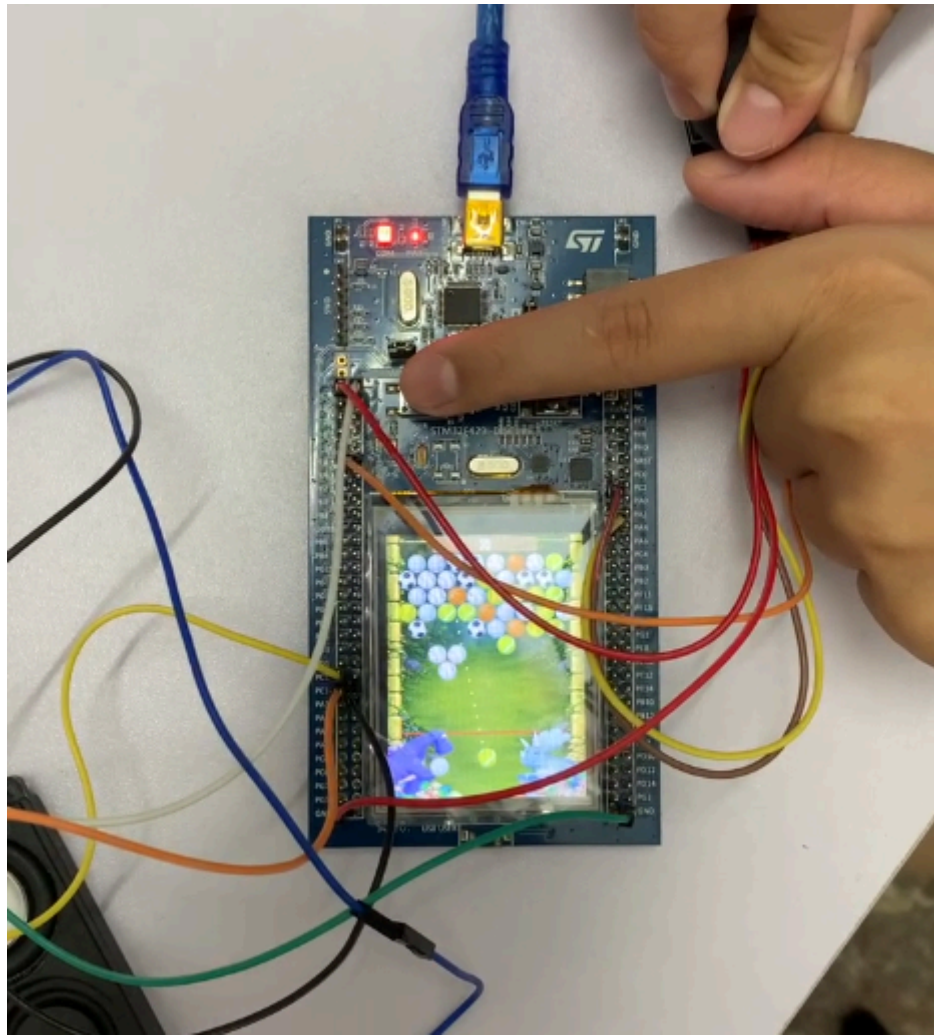
2. Demo:



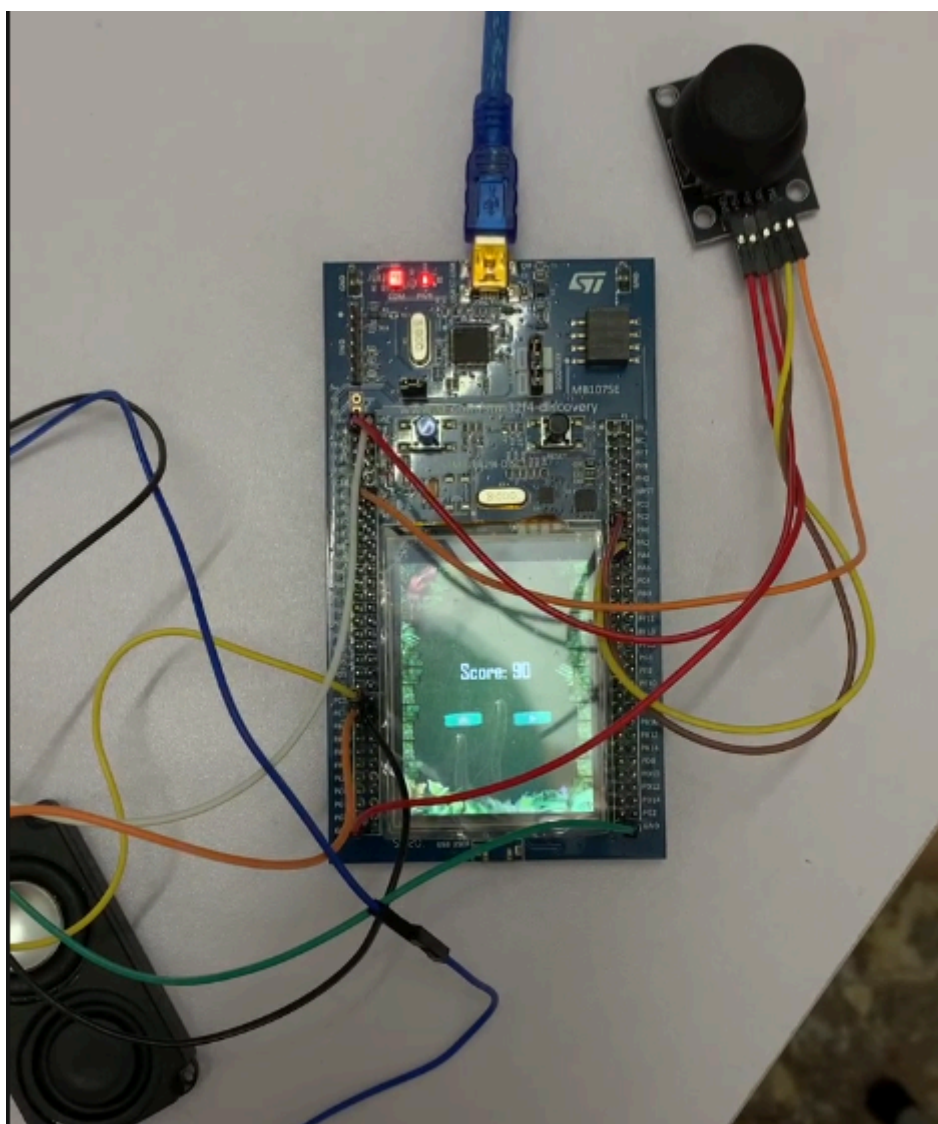
Hình 1: Màn hình Start Game



Hình 2: Màn hình chơi game



Hình 3: Hiệu ứng bóng rơi



Hình 4: Màn hình hiện điểm số

V. Kết luận:

1. Ưu điểm/ Nhược điểm của hệ thống:

Ưu điểm:

- Hiệu năng xử lý cao: STM32F429ZIT6 sử dụng vi điều khiển ARM Cortex-M4 tốc độ lên đến 180 MHz, đáp ứng tốt các tác vụ thời gian thực như xử lý logic game, điều khiển màn hình và phát âm thanh đồng thời.
- Điều khiển trực quan: Joystick cho phép người chơi điều khiển hướng bắn và thao tác bắn một cách linh hoạt, tạo cảm giác chơi tự nhiên và dễ làm quen.
- Chất lượng âm thanh tốt: Module MAX98357A sử dụng giao tiếp I2S và tích hợp cả DAC cùng bộ khuếch đại Class D, giúp âm thanh phát ra rõ ràng, ít nhiễu và đủ công suất để điều khiển loa 8 Ω .

- Thiết kế phần cứng đơn giản: Việc sử dụng các module tích hợp như joystick và MAX98357A giúp giảm số lượng linh kiện rời, đơn giản hóa quá trình lắp ráp và giảm nguy cơ lỗi phần cứng.
- Tiêu thụ năng lượng thấp: Bộ khuếch đại Class D trên MAX98357A có hiệu suất cao, tỏa nhiệt ít hơn so với các bộ khuếch đại analog truyền thống, phù hợp với các hệ thống nhúng.
- Khả năng mở rộng: Hệ thống vẫn còn nhiều chân GPIO và các ngoại vi như UART, SPI, I2C, CAN, USB..., cho phép bổ sung thêm cảm biến, nút nhấn, module Bluetooth hoặc các chức năng mới trong tương lai.

Nhược điểm:

- Phụ thuộc vào module âm thanh: MAX98357A chỉ hoạt động với giao tiếp I2S, vì vậy không thể sử dụng trực tiếp tín hiệu PWM như các module khuếch đại LM386 hoặc PAM8403.
- Giới hạn về giao diện điều khiển: Hệ thống chỉ sử dụng joystick, nút bấm và cảm ứng nên các thao tác điều khiển còn đơn giản, chưa hỗ trợ nhiều chức năng như menu nhiều cấp hoặc điều khiển đa nút.
- Chưa có khả năng lưu dữ liệu: Điểm số và trạng thái trò chơi chỉ được lưu trong RAM, sẽ mất khi tắt nguồn nếu không bổ sung bộ nhớ ngoài hoặc sử dụng Flash/EEPROM.
- Khả năng giải trí còn hạn chế: Hệ thống chỉ hỗ trợ một trò chơi đơn lẻ, chưa có các tính năng như lựa chọn cấp độ, lưu tiến trình, bảng xếp hạng hoặc chơi nhiều người.

2. Giải pháp đề xuất trong tương lai:

- Nâng cấp gameplay: Bổ sung nhiều cấp độ chơi (Level), tăng độ khó theo thời gian, đa dạng màu sắc và kiểu sắp xếp của các quả trứng để trò chơi trở nên hấp dẫn hơn.
- Hoàn thiện giao diện người dùng: Thiết kế giao diện trực quan hơn với menu chính, màn hình cài đặt, tạm dừng (Pause), kết thúc trò chơi (Game Over) và hiệu ứng chuyển cảnh nhằm nâng cao trải nghiệm người dùng.
- Bổ sung hiệu ứng âm thanh và hình ảnh: Thêm nhạc nền, nhiều hiệu ứng âm thanh cho các sự kiện trong game và các hiệu ứng đồ họa như rung màn hình, hiệu ứng nổ hoặc chuyển động mượt mà hơn.
- Lưu điểm số và tiến trình chơi: Sử dụng bộ nhớ Flash hoặc EEPROM để lưu điểm cao (High Score), thông tin người chơi hoặc trạng thái trò chơi ngay cả khi tắt nguồn.

VI. Tài liệu tham khảo:

- Slides học phần hệ nhúng (IT4210) do giảng viên - thầy Ngô Lam Trung cung cấp.
- MAX98357A Datasheet:
<https://www.analog.com/media/en/technical-documentation/data-sheets/max98357a-max98357b.pdf>
- STM32F429ZI Discovery Board User Manual:
https://www.st.com/resource/en/user_manual/um1670-discovery-kit-with-stm32f429zi-mcu-stmicroelectronics.pdf

VII. Phụ lục:

Link github dự án:

<https://github.com/huyentran2005/DinoEggShooter/tree/main/STM32CubeIDE>

List âm thanh sử dụng:

https://sounds.sprites-resource.com/pc_computer/dynomitedeluxe/asset/448285/

Link video demo:

https://drive.google.com/file/d/1tVud6UPBYtfNmHG_k0Mvs7cpGbfDLQ7/view?usp=sharing